高级语言程序设计

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2026/2/8

课程性质

- ◆JAVA与UML结合
  - ■掌握面向对象编程，理解面向对象建模
- ◆不是一般纯粹的程序设计语言课
  - ■不但讲授面向对象编程，也讲授面向对象建模
- ◆不是单纯的软件工程
  - ■会利用一些面向对象的思想

课程目标

◆朴素的OOP → 系统化的OOP

◆传统OO技术 → 现代的OO技术



课程内容

◆第一部分: Java语言基础

- ■ Java讲授Java知识体系最重要最基础的内容
- ■ 进一步提高还需要学习很多知识
  - □设计模式: 在各种应用系统中被大量应用，是设计的“常用套路”
  - □作为Java WEB开发人员，需要掌握HTML/CSS/JavaSCript和数据库相关的知识
  - □常用的应用程序框架: Spring MVC, Spirng Boot, MyBatis等
  - □Java 开发者路线图
    - ➢ https://github.com/s4kibs4mi/java-developer-roadmap/blob/master/i18n/zh-CN/ReadMe-zh-CN.md

◆第二部分：UML建模

- ■解决软件系统的分析、设计、概念表达问题。
- ■利用UML建模，协助开发者更好地设计、实现和维护Java程序。

◆第三部分: 建模的应用

- ■通过常见的设计模式例子，讲解UML建模和Java语言程序设计的实际应用。
- ■由于课时关系，这部分内容需要大家自学。

为什么学习JAVA?

Java是一种应用广泛的面相对象编程语言

计算的未来正受到Internet的深刻影响，而Java承诺将继续保持这一未来的很大一部分

Java使用户能够在Internet上为服务器，台式计算机和小型手持设备开发和部署应用程序

为什么学习UML?

软件开发的现状：大规模，复杂化；开发周期短；国际竞争；业务系统和嵌入式系统的相互融合

软件开发的课题：质量（Quality）；成本（Cost）；工期（Delivery）；生产性

面向对象的有效性：能够从用户的角度来分析系统；用户和开发人员之间能够很方便地进行交流；应对需求变化的能力强（封装）；使系统变得更容易理解，提高代码的可复用性和系统的可维护性。（泛化，封装）

第一章 JAVA 语言基础

- 1.1 Java语言概述
- 1.13 Java开发环境
- 1.2 数据类型、变量和表达式
- 1.3 关键字
- 1.4 常量
- 1.5 运算符以及分隔符
- 1.6 变量与声明
- 1.7 变量的作用域
- 1.8 表达式以及运算符的优先级
- 1.9 对象与类
- 1.10 Java的包
- 1.11 Java的代码组织
- 1.12 文本输入和输出

1.1 Java 语言概述: 1.1.1 Java 语言的发展历史

西北工業大學 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- Java的设计始于上世纪90年代初。最初被称为Oak（橡树），是由Sun Microsystems公司的工程师詹姆斯·戈斯林设计（James Gosling）。Oak语言最初是为家用电器设计的，在后来的发展中逐渐演变为用于互联网应用的通用编程语言。它的基础是当时最为流行的C和C++语言。
- 1993年，WWW（万维网）迅速发展，SUN公司发现可以利用Oak语言的技术来创造含有动态内容的WWW网页，便将改造后的Oak语言改名为Java语言 。
- 1995年5月23日，Sun Microsystems正式推出Java语言和平台。它最初被称为“Java 1.0”，并被公认为是第一个真正的跨平台编程语言，因为开发的Java程序可以在不同操作系统上运行。
- Java吸收了C++语言的一些优点，但具有更简单的语法和更强大的安全性。Java是一种面向对象的语言，具有垃圾回收机制、跨平台性、多线程支持和丰富的标准库等特点。
- Java 1.2推出最初的Java Servlet，Java在电子商务方面开始崭露头角，尤其是JSP技术的出现，使得Java成为基于Web应用程序的首选开发工具。

1.1.1 Java 语言的发展历史

- ◆持续发展：Java语言经历了多个版本的发布和演进，不断追加新的功能和改进。其中一些重要版本包括Java 5（引入泛型、枚举等新特性）和Java 8（引入Lambda表达式和Stream API）等，为Java编程带来了革命性的变化。
- ◆Oracle收购Sun Microsystems：2010年，Oracle公司收购了Sun Microsystems，从而成为Java的继续支持者和管理者。Oracle持续推动Java的发展，并发布了新的版本和更新。
- ◆2017年，Oracle修改了Java的版本发布策略。从 2018 年开始，每 6 个月就会发布一个 Java 版本。（每年3月和9月）。每大概3年发布一个LTS版本(Long-Term Support, 长期支持)
- ◆2026年2月现在，最新的Java版本是2025年9月发布的Java 25. Java 25同时也是最新的LTS版本。
  - ■目前LTS版本有：Java 8, Java 11, Java 17, Java 21，Java 25

1.1.1 Java 语言的发展历史

◆Java语言目前是最流行的编程语言之一. TIOBE编程语言排行榜（2026/2/16）：

■ https://www.tiobe.com/tiobe-index/

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		Python	22.61%	-0.68%
2	4	↑	C	10.99%	+2.13%
3	3		Java	8.71%	-1.44%
4	2	↓	C++	8.67%	-1.62%
5	5		C#	7.39%	+2.94%
6	6		JavaScript	3.03%	-1.17%
7	9	↑	Visual Basic	2.41%	+0.04%
8	8		SQL	2.27%	-0.14%
9	11	↑	Delphi/Object Pascal	1.98%	+0.19%
10	18	⇈	R	1.82%	+0.81%

1.1.2 Java 语言的特点

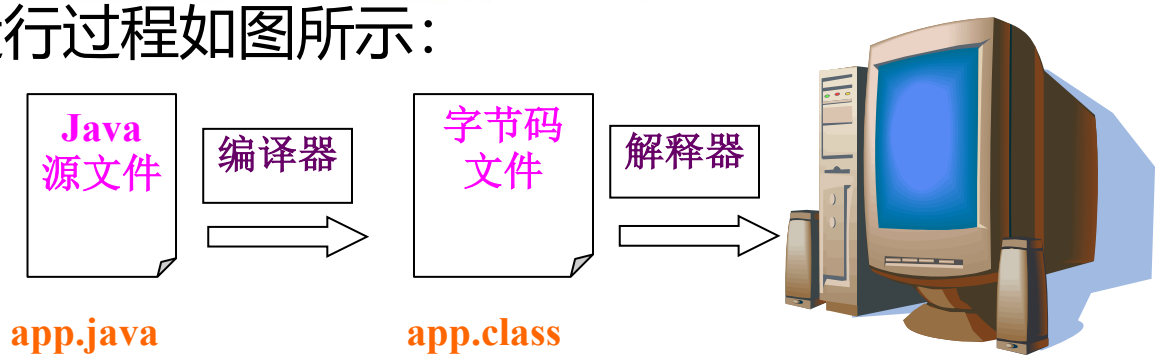
- 1 编译和解释并存：由编译器将Java源程序编译成字节码文件，然后再由Java运行系统解释执行字节码文件（解释器将字节码再翻译成二进制码运行）。字节码不是机器指令。
- 2 平台独立性和可移植性： 字节码可以在任何机器上被JVM解释执行，原始数据类型的大小与机器无关。通过这样的机制，实现了“Write Once, Run Anywhere”（WORA）的理念。
- 3 面向对象: Java是一种纯粹的面向对象编程(OOP)语言。它支持OOP的基本概念，如类、对象、继承、封装、多态和抽象。Java程序代码和数据都驻留在对象和类中。
- 4 健壮且安全：Java具有严格的运行时数据类型检查，消除了利用指针进行内存管理，代之以自动垃圾回收。通过这些手段 大大提高了程序的健壮性和安全性。
- 5 分布式: Java提供了网络上进行数据传输和远程操作的手段。
- 6 多线程: Java提供了多线程能力，允许同时执行多部分代码，提高应用程序的响应性和性能。与此相比C++没有内置的多线程机制，需调用操作系统的多线程功能来进行多线程程序设计。

◆ Java与C++的区别

- ■Java中没有#include 和#define 等预处理功能，用import语来句包含其它类和包；
- ■Java中没有structure，union及typedef；
- ■Java中没有不属于类成员的函数，没有指针和多重继承，Java只支持单重继承；
- ■Java中禁用goto，但goto还是保留的关键字；
- ■Java中没有操作符重载；
- ■Java中没有全局变量，可以在类中定义公用、静态的数据成员实现相同功能；

1.1.3 Java虚拟机

◆Java程序的运行过程如图所示：



◆字节码(bytecode)是Java虚拟机（Java Virtual Machine-JVM）的指令组。字节码最大的好处是可跨平台运行，即Java的字节码可以编写一次，到处运行。

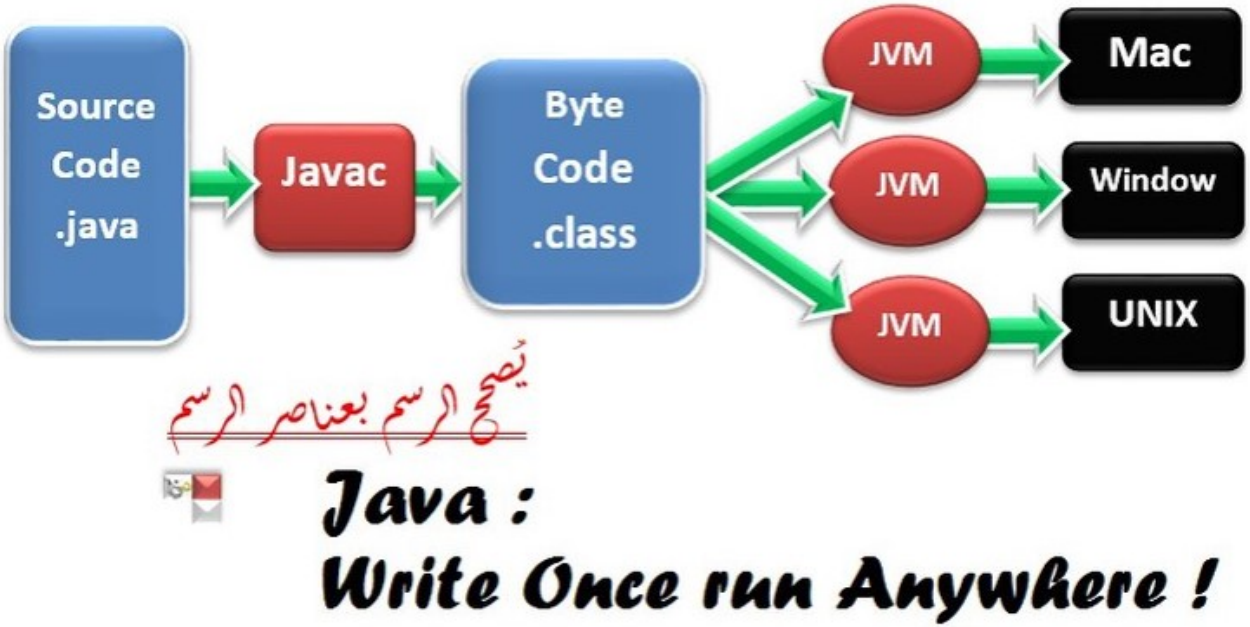
◆任何一种可以运行Java字节码的软件均可看成是Java的“虚拟机”，可以把Java的字节码看成是JVM上所运行的机器码(machine code)。

- 解释器是JVM的一部分。

◆JVM还可以解释执行Scala、Groovy、Clojure 和Processing 等新语言编译的字节码。

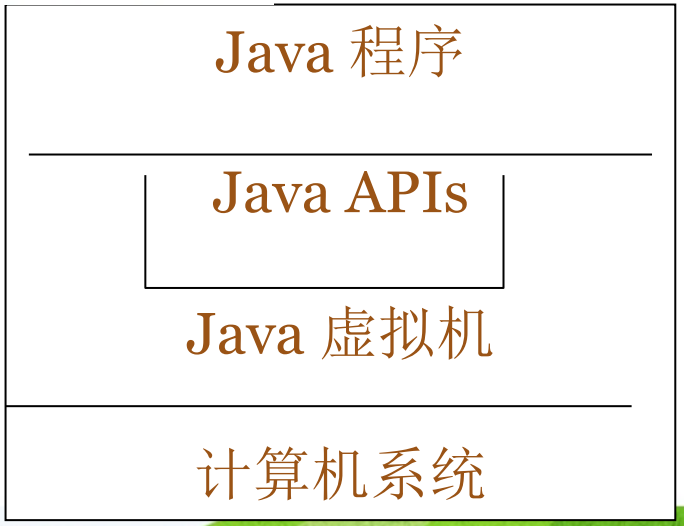
1.1.3 Java虚拟机

◆Write Once, Run Anywhere：



◆借助于JVM，实现跨平台。

◆



1.1.3 Java虚拟机

- 编译执行：可以直接把Java代码编译为本地代码（机器指令）：损失跨平台性。只有特殊情况才使用。
- 解释执行：解释器对字节码逐条翻译并执行， 相对性能较差。
- JIT和混合模式：
  - Java JIT（just in time）Compiler即时编译器是目的在于提高java程序的性能，改变人们“java比C/C++慢很多”这一印象。
  - 混合模式是Java最常用的运行模式。当JVM发现某段字节码被频繁执行时，它会使用JIT编译器将这些字节码编译成特定于平台的机器码。编译完成后，JVM会直接运行这些机器码，而非继续解释字节码，从而提升程序运行效率。
  - 在这种模式下，JVM使用解释器运行程序的大部分代码，同时使用即时编译（JIT编译器）技术来编译热点代码（经常执行的代码段）成为本地平台的机器码。这种方法结合了解释执行的快速启动时间和编译执行的高性能运行速度。
- 目前高版本的Java默认都是采用混合执行模式。
  - JIT并不一定总能提高程序的执行效率甚至适得其反，这很大一部分取决于开发人员所写的程序质量。
- http://cs-fundamentals.com/java-programming/difference-between-jdk-jre-jvm-jit.php
- https://www.cnblogs.com/lyhero11/p/5080306.html

1.1.4 面向对象编程概述

◆自顶向下编程

- 1970、80年代，软件工程的主要方法是结构化程序设计。主要思想是：对于一个大问题的求解，首先将大问题分解为多个小部分；然后对于每一个部分，把它当作一个新问题，再分解成更小的部分；最终，解决所有的小问题。这种方法称为自顶向下编程。
- 这种方法认为业务逻辑是主要的，数据结构的次要的，在业务设计过程中补充数据结构，这种方法的缺点是数据考虑不足。另外，由于数据的不兼容，代码复用比较困难。

◆自底向上编程

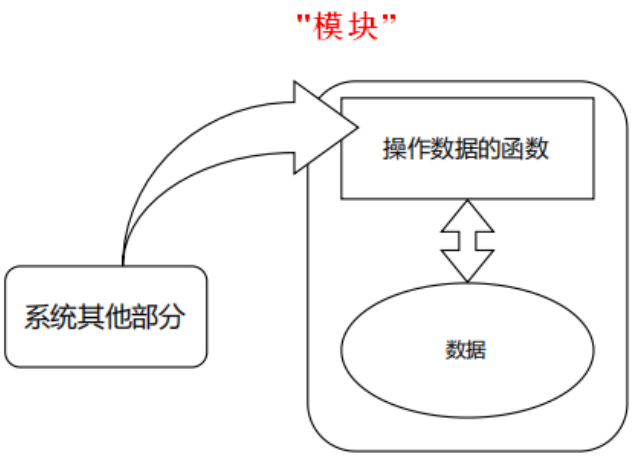
- 在实践中经常结合自顶向下和自底向上方法相结合。
- 自底向上的主要思路，是先从一个一个比较基础的问题解决着手。把这些问题的解决封装为“模块”，然后逐步构筑完整的系统。
- “模块”本身是一个可复用的部件，可以“plug in”到系统中。与系统其他部分的的交互简单

、明确、直接。

- 只要模块正确地履行了其分配的角色，模块内部发生的细节对于整个系统来说并不重要。这被称为信息隐藏（information hiding），它是软件工程中最重要的原则之一。

◆自底向上编程

- 模块由数据和代码构成。数据隐藏在模块内
- 使用模块的程序只能通过调用模块提供的函
  存取数据。
- 对使用者来说不必担心数据保存的细节。
- 这种方法最终发展为面向对象编程（OOP）



◆对象

- 面向对象编程的核心概念是对象，它是一种包含数据和函数的模块。
- OOP 的观点是对象是一种“自治”的实体，它具有内部状态（它包含的数据）并且可以响应消息（对其函数的调用）。
- 例如，邮件列表对象具有由姓名和地址列表组成的状态。如果向它发送一条消息，告诉它添加名称，它将通过修改其状态以反映对更改的响应。如果向它发送一条消息，告诉它输出信息，它会通过输出其名称和地址列表来响应消息。

◆封装(Encapsulation)

- 面向对象编程（OOP）中的封装是一种将数据（属性）和代码（方法）绑定到一起的编程技术，这样的结构称为一个对象。封装的关键是确保对象的内部状态和行为的安全，防止外部代码直接访问并修改对象内部的数据和实现细节。

◆OOP程序的构成

- OOP 编程的第一步是识别问题领域中涉及的对象以及这些对象应响应的消息，即程序运行是一组对象以及它们之间的交互。每个对象都有自己的数据和自己的职责，对象通过相互发送消息进行交互。
- OOP 程序往往是世界本身运作方式的一种模拟，因此更容易编写程序并且更容易理解程序的执行

◆多态性（polymorphism)

- 对象“知道”如何响应某些消息。不同的对象可能以不同的方式响应相同的消息。
- 例如，一条“输出”消息会产生不同的结果，具体取决于它发送到的对象。对象的这种特性，不同的对象可以以不同的方式响应相同的消息，被称为多态性。

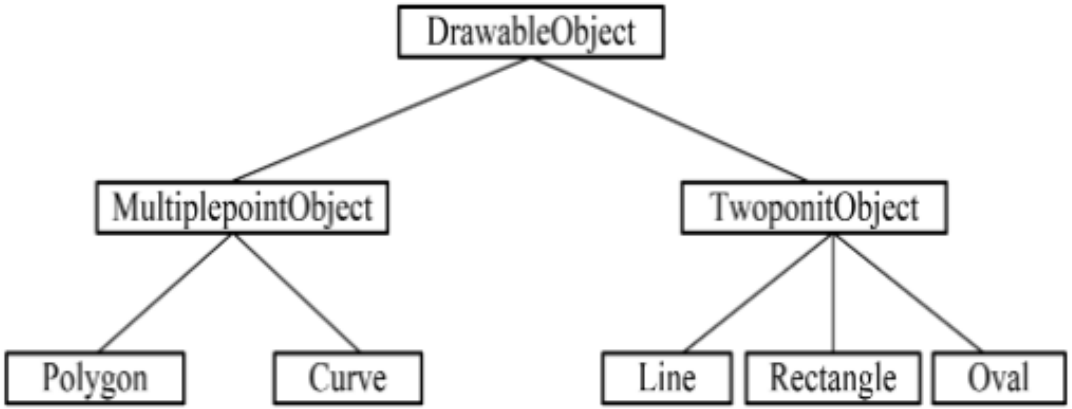
◆类

- 现实世界的实体之间具有一种“相似性”，即它们之间包含相同类型数据并以相同方式响应相同消息，这些实体可以归纳为同一类，称为面向对象的抽象。
- 例如，对于一个可以绘制直线、矩形、椭圆、多边形和曲线的绘图程序。在程序中，绘图窗口上的每个可见对象都可以由程序中的一个对象来表示。这样绘图工具生成的图形包含大量的直线、矩形、椭圆、多边形和曲线等对象，但是这些对象抽象为五类型，每种类型的对象都可以被绘制。例如，所有的线都属于一个类，所有的矩形都属于另一个类，依此类推。

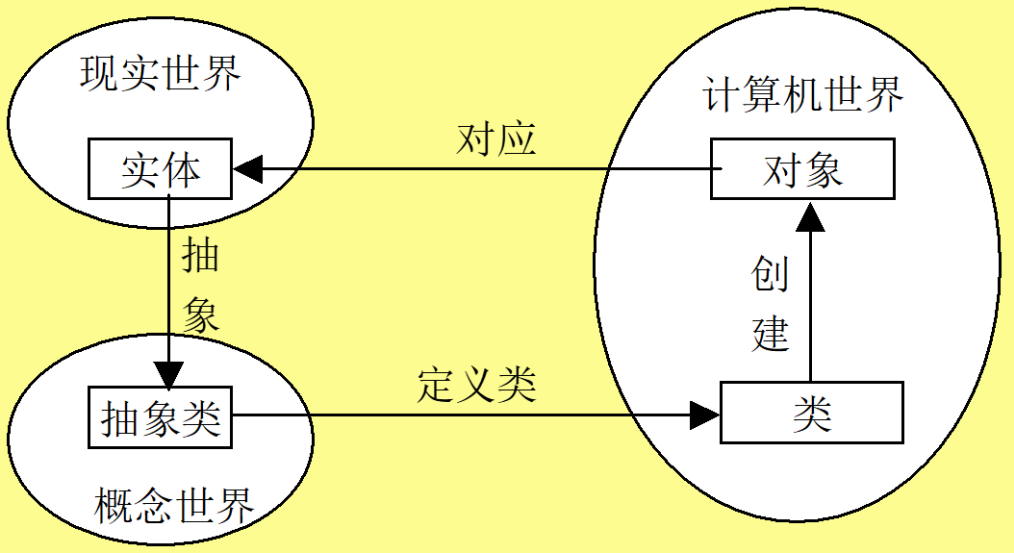
这样的类就是对大量对象的一种抽象。

◆继承

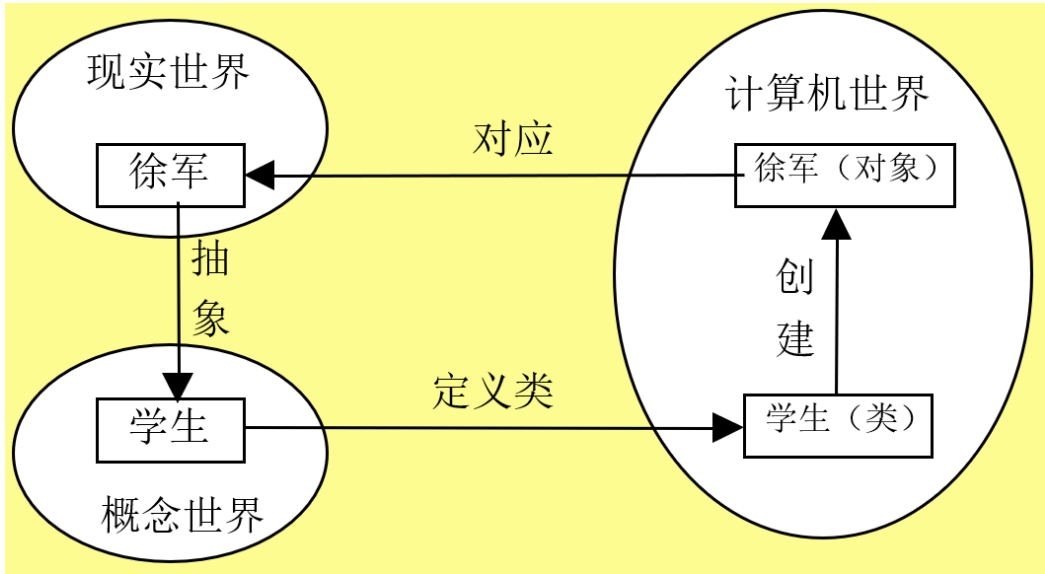
- 上面的类显然是相关的，它们都代表“可绘制对象”。
- 例如，它们都能够响应“drawable objects”的信息。为了体现相关对象的相同点和不同点，可以根据某个特征对这些对象进行一个层次上的分类。层次上的分类依据每种类型的对象所需的数据特征，即：可以将多边形和曲线组合在一起作为“多点对象”，而直线、矩形和椭圆是“两点对象”。例如，一条线由它的两个端点决定，一个矩形由它的两个角所在的点决定，一个椭圆由包含它的矩形的两个角对应的点决定，所以它们按照层次定义一个两点对象。因此，可以将 5 种图形类在层次上分类后形成下图2 所示的结构。
- 这些类的关系称为继承，类的子类被称为继承了该类的所有属性和方法。子类可以添加到它的继承中，它甚至可以“覆盖”该继承的一部分（通过定义对某些消息的不同响应）。尽管如此，线条、矩形等都是可绘制的对象，而 DrawableObject 类表达了这种“覆盖”关系。
- 继承是组织程序的有力手段，它还与重用软件组件的问题有关。类是最终的可重用组件，它不仅可以直接重用，而且可以通过定义一个子类并只进行必要的小改动以使其完全适应一些新的需求来重用它。



1.1.4 面向对象编程概述



- 现实世界中概念，抽象为类。
- 现实世界的实体，对应为对象。



1.1.5 基本 Java 应用程序

◆一个基本的Java程序如下，它在屏幕输出”Hello,World!”

- ■消息输出是调用了一个Java的“内置函数”System.out.println()完成的。
- ■类名字为HelloWorld
- ■程序入口为main函数
- ■main前面的public意味着可以从外部调用

◆运行方法

- ■javac HelloWorld.java (生成HelloWorld.class文件)
- ■Java HelloWorld

```
public class HelloWorld {
    public static void main(String[] args) {
    System.out.println("Hello World!");
    }
}
```

对于熟悉C语言的同学，这样也可以：
System.out.printf("Hello %s", "World!");

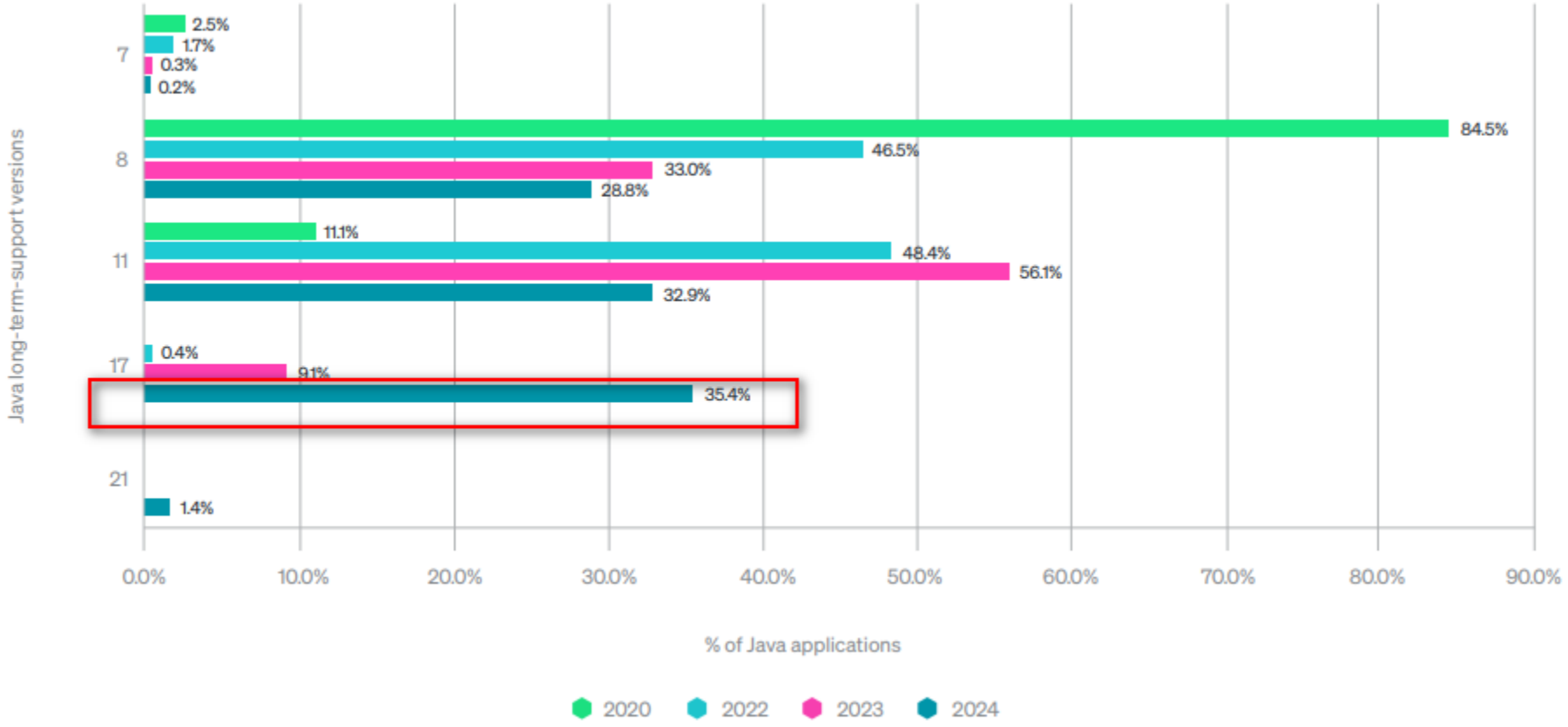
◆Java程序的基本构成

```
<包声明>
<导入类>
public class <program-name> {
<变量声明或子函数>
public static void main(String[] args) {
<statements>
}
<变量声明或子函数>
}
```

- 2025年9月，Java 25发布。它是最新的LTS(Long Term Support)版本
  - 今后每六个月进行一次新特性发布，每季度进行一次更新发布，每三年进行一次LTS（长期支持）发布。
  - 最新的JDK是 JDK 25.
- 目前主要有Oracle的JDK和OpenJDK两个版本
  - Linux系统上一般使用Open JDK
- 不管版本怎么变动，本课程学习的最基础的内容都很重要

◆流行的java版本: java 8, java 11, java 17

■ https://newrelic.com/sites/default/files/2024-04/new-relic-state-of-the-java-ecosystem-report-2024-04-30.pdf

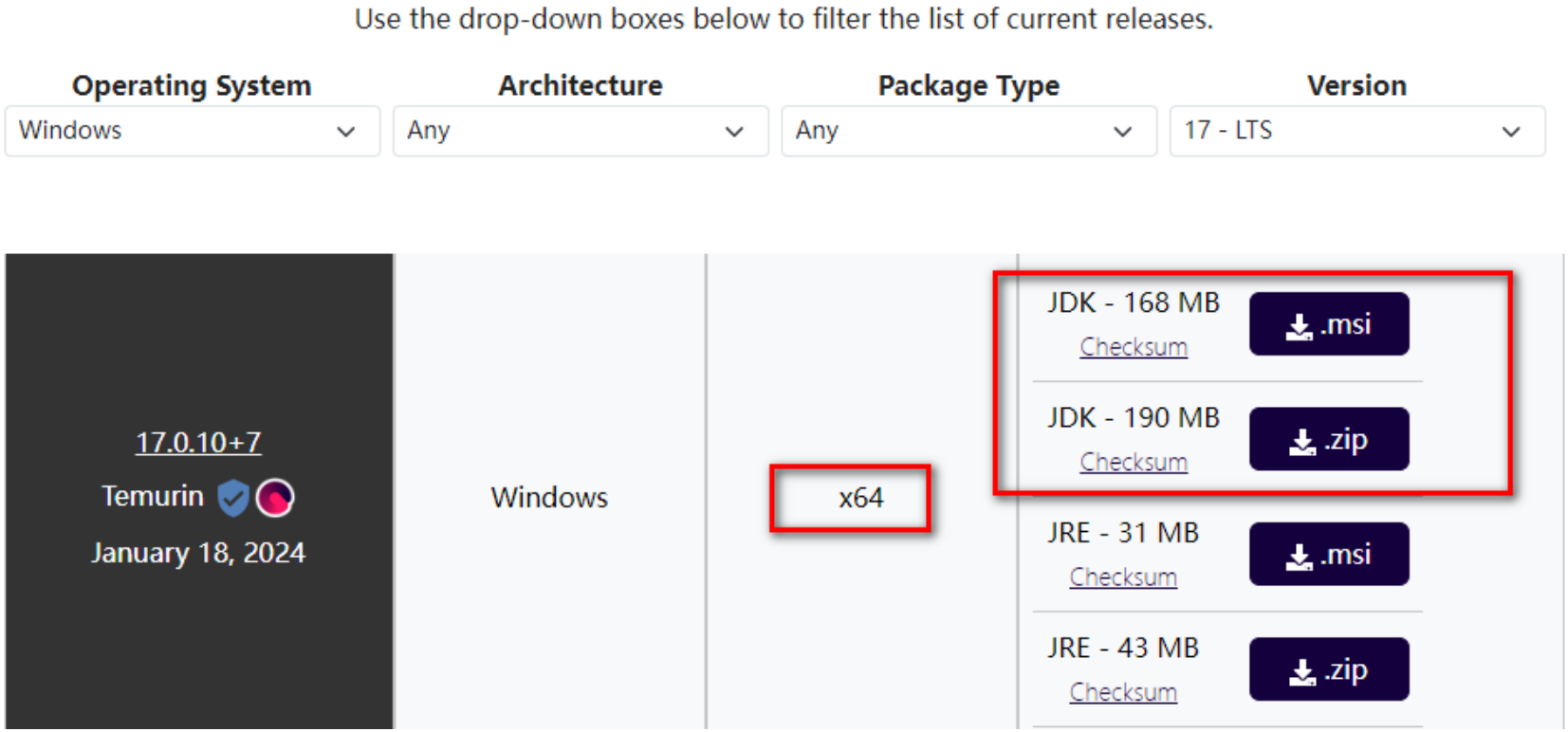


1.13 Java开发环境

- JDK（Java Development Kit）是Java编程语言的开发工具包，包括编译器（javac）、Java运行时环境（JRE）、调试器等工具。它为开发人员提供了创建、编译和调试Java应用程序的环境。
- JRE（Java Runtime Environment）是在计算机上运行Java应用程序所必需的软件包。它包含Java虚拟机（JVM）和Java类库，用于执行Java程序。在没有JDK的情况下，只需要JRE就可以运行Java应用程序。
- Open JDK
  - 从Java 11版本开始，Oracle对商业用途的Java产品实施了新的许可政策（付费），因此很多项目采用Open JDK.
  - Open JDK有多个发行版，最流行的是AdoptOpenJDK（现在称为Adoptium）
  - 建议使用 Open JDK 8或者17。作为学习也可以使用Oracle的JDK或者Open JDK.

◆下载方法

- https://adoptium.net/temurin/releases/?os=windows
- MSI是Windows 安装包，使用简单
- ZIP是绿色版，需要解压后自己设置环境变量



◆编译和运行Java应用程序

```
public class App2_1{
   public static void main(String[] args)  {
      System.out.println("Hello Java!");
   }
}
class App2_1_2{
   public static void main(String[] args)  {
      System.out.println("Hello Java from App2_1_2");
   }
}
```

```
>javac App2_1.java (编译)
>java App2_1       (执行)
>java App2_1_2     (执行)
```

2.2 JDK参考文档下载与安装

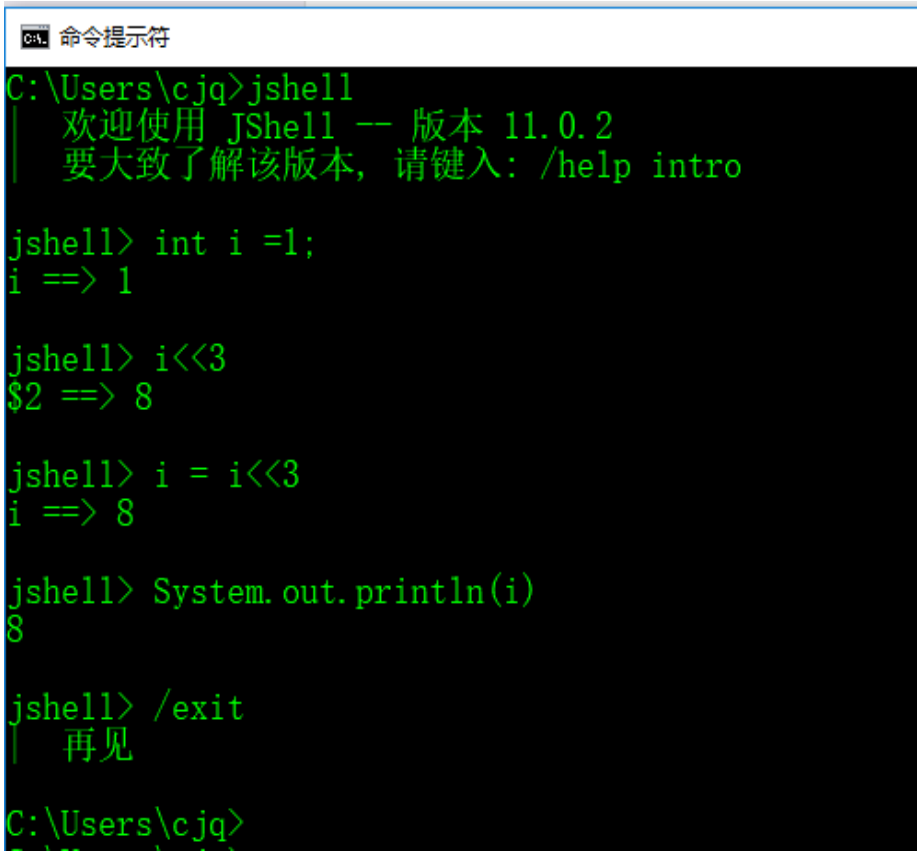
- 输入网址 http://www.oracle.com/technetwork/java/javase/downloads/index.html
- 进行JDK参考文档的下载，取得JDK参考文档的安装文件
- 帮助文档的使用
  - 可以使用HTML的帮助
  - 也可以查CHM格式的帮助（JDK8)
  - 在开发中要学会及时查帮助。

1.13 Java开发环境

- **Eclipse(免费， 推荐。)**
  - MyEclise(收费)
- **Intelij IDEA(收费)，当前最流行的工具。**
  - **有学生免费版可以申请。**
  - **建立版权意识，工作中避免使用盗版。**
- NetBeans(免费)
- NotePad (Windows记事本)
- NotePad++(免费)
- VS Code (免费)

JShell

- Java Shell, Java 9 新特性。是一个 REPL（Read-Eval-Print Loop，读取-求值-输出 循环）工具。
  - 类似于python解释器或者BASIC解释器，允许交互式执行Java代码。
- JShell 可以帮助你快速的测试代码。你可以测试单个语句、测试使用不同的参数调用方法，也可以在一个 JShell 会话中测试不熟悉的 APIs。
- 但是，JShell 并不是 IDE 的替代品。当你开发应用时，可以粘贴代码到 JShell 并测试它，然后把测试通过的代码粘贴到程序编辑器或者 IDE 中。



◆1.2.1 Java 符号集

Java 的符号分为如下 5 种类型

- ■标识符（identifies)
- ■关键字（keywords)
- ■常量（literals)
- ■运算符（operators)
- ■分隔符（Separator)

所有的 Java 程序均由以下的信息构成：标识符、关键字、常量、运算符、分隔符，以及不属于 Java 符号的注释。

◆1.2.2 注释

■Java 支持 3 种形式的注释。

表 1.1

注释类型	描述
//注释内容	所有从//到行末的字符将被忽略
/*注释内容*/	所有在/*和*/中的字符将被忽略。这些注释能扩展到多行
/**注释内容*/	所有在/**和*/中的字符将被忽略。但是这些注释只应用于声明之前，因为他们将被 javadoc 工具用于建立自动文档。

- 设计 Java 程序时，给程序的一些主要功能增加注释将有助于后续的代码改进或者提高代码的易读性。注释不仅仅有助于自己对程序的修改和维护，而且也有助于别人阅读代码。
- 此外，注释也一定程度增强了代码的可重用性。

◆1.2.3 标识符

- ■可以由字母、数字、下划线(—)、美元符号($)组合而成。
- ■必须以字母、下划线或美元符号开头，不能以数字开头。
- ■关键字不能当标识符使用。
- ■标识符是大小写敏感的
- ■所有 16 进制值大于 00C0的 Unicode 码字符。比如希腊字母，日语，汉字都可以。（不提倡这么用）。

◆标识符的拼写看似很简单，但是标识符的使用需要遵守编码风格。标识符书写的规范，源程序的可读性就大大增强，也能够减少程序错误。以下是标识符书写的启发式规则。

①标识符不宜过短，过短的标识符导致程序的可读性差，因此，应避免使用类似 a，b这样的不具有任何含义的标识符；另一方面，标识符也不宜过长，否则将增加录入工作量，增加出现错误的可能性。

②尽量减少使用除下划线、美元符号和字母“a”-“Z”之外的其它字母，以减少录入时的工作量。

③除非有特殊原因，最好只使用下划线，少使用或者不使用美元符号。这里面的原因之一是当链接 c 代码时，美元符需要在库例程中使用。

④标识符开头以及标识符中出现的每个单词的开头字母大写，其余字母全部采用小写，如 HelloWorld。

◆一些大的企业或者组织都有自己的编码规范

◆1.3 关键字

又叫保留字，Java本身使用的标识符，不能另作他用。

1. abstract	9. class	17. extends	25. import	33. private	41. switch	49. volatile
2. assert	10. const (不使用)	18. final	26. instanceof	34. protected	42. synchronized	50. while
3. boolean	11. continue	19. finally	27. int	35. public	43. this	
4. break	12. default	20. float	28. interface	36. return	44. throw	
5. byte	13. do	21. for	29. long	37. short	45. throws	
6. case	14. double	22. goto (不使用)	30. native	38. static	46. transient	
7. catch	15. else	23. if	31. new	39. strictfp	47. try	
8. char	16. enum	24. implements	32. package	40. super	48. void	

- const和goto虽然保留，但是没有使用。
- true、false和null虽然是Java中的保留字，但严格来说它们并不是关键字，因为它们代表的是字面量值，而非语言结构的一部分。

◆Java的基本数据类型如下，这些基本数据型都有对应的字面量。

数据类型	类型名	占用位数（字节数）	取值范围	默认值
布尔型	boolean	8(1 byte)	true,false	false
字符型	char	16(2 bytes)	'\u0000'（0）～'\uffff'（65535）	'\u0000'
字节型	byte	8(1 byte)	-128（-2^7）～127（2^7-1）	0
短整型	short	16(2 bytes)	-32768（-2^15）～32767（2^15 - 1）	0
整型	int	32(4 bytes)	-2,147,483,648（-2^31）～2,147,485,647（2^31 - 1）	0
长整型	long	64(8 bytes)	-9,223,372,036,854,775,808（-2^63）～9,223,372,036,854,775,807（2^63 -1）	0
浮点型	float	32(4 bytes)	约1.4E-45～3.4028235E38即2^(-149)～(2-2^(-23))*2^127	0.0f
双精度型	double	64(8 bytes)	约4.9E-324～1.79769313486231570E308即2^(-1074)～(2-2^(-52))*2^1024	0.0d

- 在Java中还有引用数据类型（复合数据类型）（reference）
- 引用数据类型在内存中存放的是指向该数据的地址，不是数据值本身。
  - 类、数组、接口等。
  - 引用数据类型的默认值是null.

◆类的成员变量有默认值

■--这些变量的存储空间通过new 在堆中分配。分配时new操作本身对分配的内存区域首先全部清零。

◆方法局部变量没有默认值

■--这些变量的存储空间分配在栈中。随着方法的调用和返回而分配和释放，在这个过程中，无法频繁地进行清零操作，否则大大影响执行效率。

1.4 常量（字面量）

◆整数：

■int有十进制，八进制，十六进制三种写法，比如: 63, 077, 0x3f

■long型长度为64bit,

比如：long a = 100000L，long b = -200000L；小写L也可以，但是容易和数字1混淆，一般不这么写。

■short和byte: 节约存储空间时候可以用这两种类型。

比如: short s=1000;　byte b=126;

◆Java定义了整型最大值和最小值的常量Integer.MIN_VALUE，其中Integer为”引用数据类型”

- ◆浮点数
  - ■float: float f1 = 314.5f。Float字面量末尾需要写上f或者F
  - ■double d=3.145或者 double d=3.145D (小写d也可以)
  - ■可以使用科学计数法。
- ◆布尔型
  - ■boolean flag = true
  - ■boolean flag = false

◆字符常量

■单引号内的字符。Java的字符编码采用16位Unicode.

比如: char ch='5'; char ch2='国'; (一个unicode字符)

■必要时可以进行字符转义

描述	标准表示方法	字符序列
Continuation	<new line>	\
换行符	NL(LF)	\n
水平制表符	HT	\t
退格符	BS	\b
回车符	CR	\r
进纸	FF	\f
反斜线	\	\\
单引号	‘	\’
双引号	“	\”
八进制位模式	0ddd	\ddd
十六进制位模式	0xdd	\xdd
Unicode 字符	0xdddd	\udddd

◆字符串常量

- ■双引号内的字符序列。Java为字符串提供了类型String,不需要像c++那样把字符数组用作字符串。
- ■字符串内也可以转义：比如:System.out.println(“Sir,\nwhere are you?”)

Unicode字符采用\u0000 ----\uFFFF之间的十六进制表示。
Unicode字符表的前128个刚好是ASCII表。

◆分隔符如下表。

■此外空白符包括空格、回车、换行和制表符（Tab键），可以用来调整代码格式。

表 1.7 分隔符

符号	名称	说明
( )	圆括号	提高了括在其中的运算的优先级
{ }	花括号	定义一个作用域的作用。例如：类作用域；函数作用域；语句块作用域；静态块作用域；实例初始化块作用域；局部作用域
[ ]	方括号	数组专用
;	分号	代表一个语句的结束
,	逗号	语句分割。例如，函数参数之间分割、for 循环语句之间分割、表达式之间分割
.	圆点	对象成员的引用
...	省略号	标识可变数量的参数
@	at 符号	注解。如@Override、@Deprecated、@SuppressWarnings、@SafeVarargs、@FunctionalInterface
::	双冒号	与 lambda 表达式相关的方法引用

1.5 运算符以及分隔符

- 运算符的列表见教材“表1.8”。
- 算术运算操作符：+，-，*，/，%，++，--
- 关系操作符：>, >=, < ,<=, ==, !=
- 位操作符：>>, <<, >>>, & , |, ^(逐位异或)，~ (按位取反)
- 逻辑操作符：&, |, !, ^(异或),&&，||
- 赋值操作符：=，+=，-=，*=，/=，%=，&=，|=，^=，<<=, >>=, >>>=
- 其它操作符: ?: , [], . , ( ),(type), new, instanceof

以下是关于运算符的具体说明：

◆逻辑操作符与位操作符相同的，根据操作数判定是何种运算符

■&，|： 不允许布尔型和整型混合运算。

◆&&,|| -- 称为短路与、或。比如以下代码输出结果为:

i=1,j=0

```
public class Test2023 {
  public static void main(String[] args) {
    int i=0,j=0;
    if ((i=1)<0 && (j=2)>1) {
      System.out.println("true");;
    } else {
      System.out.println("i="+i+",j="+j);
    }
  }
}
```

◆&, | -- 称为不短路与、或；比如以下代码输出结果为:

```
i=1,j=2
```

```
public class Test2023 {
  public static void main(String[] args) {
    int i=0,j=0;
    if ((i=1)<0 & (j=2)>1) {
      System.out.println("true");
    } else {
      System.out.println("i="+i+",j="+j);
    }
  }
}
```

◆类型比较运算符 instanceof:检查一个对象实例是否是特定类或其子类的实例

■例如： e instanceof Point //Point 是一个类

1.5 运算符以及分隔符

- Java的String类型可以用+连接。这是Java唯一的“运算符重载”
  - String s="Hello"+" World";
  - 而且不光两个字符串可以相加，还可以加其他任意类型
  - String s="Hello"+5;
- >> ：带符号右移, 符号位扩展: 1010 … >> 2 结果是111010…
- >>>：无符号右移，以0 填充: 1010 … >>> 2 结果是001010 …
- 强制类型转换的一般形式：(type) expression, 例：(float)x/2
  - 对强制类型转换的限制：整型与浮点型可以相互转换，但基本类型 和数组、对象等复合类型之间不能互相转换

1.6 变量与声明

◆在 Java 语言中存储一个数据，必须使用一个变量，变量的使用包含两项内容：数据类型，标识符。

◆变量声明的格式: type> <identifier [,identifier]>;

◆可以在声明变量同时初始化。

如：int i, j=0;

```
i = 8;
float k;
k = 3.6f;
```

◆Java 语言程序中可以随时定义变量，不必集中在执行语句之前。

1.6.5 数组类型的声明

◆在 Java 语言中，数组是一种专门的类型，它们通过索引方式组织一个对象列表。另外，Java 中的数组还可以嵌套。Java 数组声明可以采用与 C 语言类似的形式，如下面的例子：

```
int count[]; //一维数组
char ch[][]; //二维数组
float f[];
```

◆此外，也可以用另外一种等价的形式。下面语句声明的数组与上面是一样的。

```
int[] count; //一维数组
char[][] ch; //二维数组
float[] f;
```

1.6.5 数组类型的声明

- Java 数组声明时并不需要指定数组的大小，这是因为声明数组时并不需要为数组分配存储空间。要分配数组的存储空间，必须使用 new 运算符，在后面的章节将详细讲述数组及相关的运算符。
- 数组是引用类型。非基本类型声明时不直接开空间，而是开辟引用空间，数据空间没有开。只有通过实例化(new)，才能给数据开辟空间。

1.6.6 引用变量的声

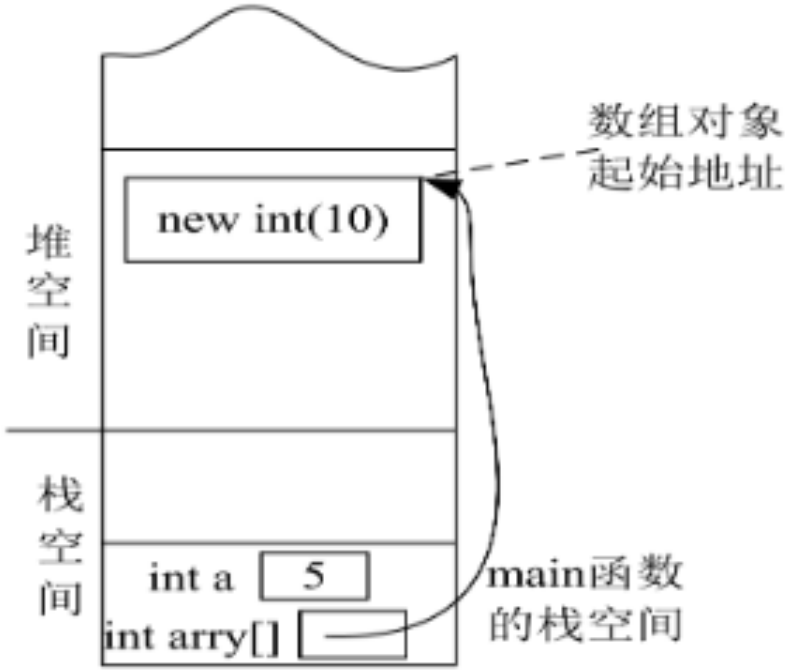
◆引用变量与基本数据类型的变量的最大区别是变量空间中存储的内容不同，基本数据变量直接存储变量的值，而引用变量存储的是某一个对象的引用。

```
class App{ //类 App 块的开始
  public static void main(String args[]){ //main 函数开始
    int a=5;
    int[] arr=new int[10];
    arr[0]=1;
    System.out.println("arr[0]="+arr[0]+",arr[1]="+arr[1]);
  } //main 函数结束
}
```

1.6.6 引用变量的声

◆对于 mian 函数的基本数据变量 a，其变量空间直接存储整数值 5

◆而对于引用变量 arr，其变量空间存储的是一个引用值，它指向堆空间的数组的起始位置（new int[10]的含义是在堆空间分配一个大小为10的数组



1.6.6 引用变量的声

◆代码执行的结果是：

```
arr[0]=1,arr[1]=0
```

◆new分配数组时，数组元素被以默认值初始化，也就是0.

1.7 变量的作用域

- 作用域，指的是标识符在程序的某一个部分有效。一个变量的作用域从该变量的定义处开始，到它所在的块结束为止。块由一对大括号“{ }”定义
  - 类成员变量的作用域为整个类。
  - 方法局部变量的作用域为它所在的代码块

1.7 变量的作用域

◆特殊情况下，一个变量可以被其它变量屏蔽，或称为“隐藏”（hidden）。

- ■类中方法的局部变量，会隐藏类的成员变量。
- ■比如下面代码输出结果为6.

◆与C复合语句不同的是：Java语言不允许在两个嵌套的复合语句内声明两个同名的变量。

```
class App{ //类 App 块的开始
  static int j=5;
  public static void main(String args[]){ //main 函数开始
    func();
  } //main 函数结束
  public static void func() {
    int j=6;
    System.out.println("j="+j);
  }
}
```

1.8 表达式及运算符的优先级别

◆表达式是由变量、操作符以及方法调用所构成的结构。以“;”结尾的一段代码，即为一个表达式。比如：System.out.println(5); 也是一个表达式。

表 1.9 运算符优先级

类别	操作符	关联性
后缀	() [] . (点操作符)	左到右
一元	++ -- ！ ~	从右到左
乘除	* / %	左到右
加减	+ -	左到右
移位	>> >>> <<	左到右
关系	> >= < <=	左到右
相等	== !=	左到右
按位与	&	左到右
按位异或	^	左到右
按位或	|	左到右
条件与	&&	左到右
条件或	||	左到右
条件	？：	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= |=	从右到左
逗号	,	左到右

- 赋值表达式会产生”副作用“：
  - int i=5; int j=(i++)+i;
  - 由于Java编译器会按照固定的顺序处理表达式，所以在单线程情况下结果是确定的，不像C语言依赖于编译器。
  - 并发情况下，当多个线程可能同时修改同一个变量时，结果会变得不可预测。
  - 即使结果是确定的，这种代码风格还是容易让人产生混淆，不易维护，还是要避免。

1.8 表达式及运算符的优先级别

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- 每个表达式都有类型
- 如果表达式的类型对于上下文不合适
  - 有时可能会导致编译错误
  - 有时语言会进行隐含类型转换

1.8 表达式及运算符的优先级别

- 扩展转换
  - byte, char, short, int, long, float, double



  - 从一种整数类型到另一种整数类型，或者从float到double的转换不损失任何信息
  - 从整数类型向float或double转换，会损失精度
- 窄化转换
  - double, float, long, int, short, byte,char



  - 窄化转换可能会丢失信息

1.8 表达式及运算符的优先级别

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- 赋值转换
  - 将表达式类型转换为制定变量的类型
- 方法调用转换
  - 适用于方法或构造方法调用中的每一个参数
- 强制转换
  - 将一个表达式转换为指定的类型
  - 例如 (float)5.0
- 字符串转换
  - 任何类型（包括null类型）都可以转换为字符串类型
  - 只当一个操作数是String类型时，适用于+运算符的操作数

1.8 表达式及运算符的优先级别

- 数字提升

  将算术运算符的操作数转换为共同类型

  - **一元数字提升**
    - 如果一个操作数是 byte, short, 或 char类型, 一元数字提升通过扩展转换将它转换为int类型
  - **二元数字提升**
    - 二元数字提升作用在特定操作符的操作数上

      *, /, %, +, -, <, <=, >, >=, ==, !=, &, ^, | and?:
    - 在必要时使用扩展转换来转换操作数类型

1.8 表达式及运算符的优先级别

- **强制类型转换**

如果要将较长的数据转换成较短的数据时（不安全），就要进行强制类型转换，否则发生编译错误。强制类型转换的格式如下：

（欲转换的数据类型）变量名

1.8 表达式及运算符的优先级别

- 字符串型数据与整型数据相互转换
  - 字符串转换成数值型数据（int，double，short等类似）

```
String MyNumber = “1234.56”;
float MyFloat = Float.parseFloat(MyNumber);
```

  - 数值型数据转换成字符串

在Java语言中，字符串可用加号“+”来实现连接操作。所以若其中某个操作数不是字符串，该操作在连接之前会自动将其转换成字符串。所以可用加号来实现自动的转换。

```
int MyInt=1234；                    //定义整形变量MyInt
String  MyString=""+MyInt；     //将整型数据转换成了字符串
```

◆类与对象

- ■在程序中，对象是通过一种抽象数据类型来描述的，这种抽象数据类型称为类(Class)
- ■一个类是对一类对象的描述。类是构造对象的模板
- ■对象是类的具体实例（instance)

1.9.1 类类型的声明

◆类的声明形式

[public] [abstract | final] class 类名称

[extends 父类名称]

[implements 接口名称列表]

{

变量成员声明及初始化;

方法声明及方法体;

}

1.9.1 类类型的声明

◆关键字

- ■class: 表明其后声明的是一个类。
- ■extends：如果所声明的类是从某一父类派生而来，那么，父类的名字应写在extends之后
- ■implements：如果所声明的类要实现某些接口，那么，接口的名字应写在implements之后

◆修饰符

- ■可以有多个，用来限定类的使用方式
- ■public：表明此类为公有类
- ■abstract：指明此类为抽象类
- ■final：指明此类为终结类

1.9.1 类类型的声明

◆类声明体
- ■变量成员声明及初始化
  - □可以有多个
- ■方法声明及方法体
  - □可以有多个

```
class ATypeName{
   /*成员变量和方法体*/
}
```

◆比如上面定义了一个新的类"ATypeName",然后就可以创建这个类的一个对象：

```
ATypeName a = new ATypeName();
```

◆但是上面语句中的对象“a”，还不能完成任何的任务，也没有自己的状态，因为还没有定义它的成员变量和成员方法。

1.9.2 定义类中的属性

■声明格式

[public | protected | private]

[static][final][transient] [volatile]

变量数据类型 变量名1[=变量初值],

变量名2[=变量初值], … ;

```
class DataOnly {
  int i;
  double d=5.;
  boolean b;
}
```

■格式说明

- public、protected、private 为访问控制符
- static指明这是一个静态成员变量
- final指明变量的值不能被修改
- transient指明变量是临时状态(不必持久化(serialize))
- volatile指明变量是一个共享变量

■显式初始化: 成员变量声明中可以包含简单的赋值表达式，这种在声明成员变量时对变量的显式赋值称为显式初始化

■每个对象都为其属性保留自己的存储空间；普通属性不在对象之间共享。

1.9.3 生成对象

- 对象是一种变量，它符合变量声明的规范。以下的例子是声明一个对象变量。
  - DataOnly data;
- data 是一个变量，它的类型是类 DataOnly，这样的变量称为引用变量。引用变量声明后，没有任何实际意义，因此需要将其指向一个实际的对象。
- 一个对象可以使用 new 来创建，通过将对象引用到具体的变量，就可以通过变量来访问对象。例如，以下代码就将对象指向一个引用变量：
  - data = new DataOnly();
- 实际应用中，可以将变量定义与对象的应用合并成一条语句，即：
  - DataOnly data = new DataOnly();

1.9.3 生成对象

- 通过引用变量，可以为对象的字段赋值。引用对象的成员赋值是通过声明对象引用的名称，后跟一个“.”（点），然后是对象内部成员的名称来完成的：
  - objectReference.member
- 例如：
  - data.i = 47;
  - data.d = 1.1;
  - data.b = false;

1.9.3 生成对象

◆可以生成一个类的多个对象。它们的存储空间是隔离的。

- DataOnly data1, data2
- data1=new DataOnly(); data2 = new DataOnly();
- data1.i=47; data2.i=48;

◆对象如果还包含其他对象，并且需要修改其数据项，只需继续使用“.”来实现。例如：

- myPlane.leftTank.capacity = 100;

◆上面的例子只包含数据成员，没有方法成员。为了使对象具有行为，就需要声明成员函数（也称为方法）



1.9.4 成员函数的定义

◆c/c++中函数，在Java中一般称作”方法"(method).

◆方法定义类的行为，确定对象可以接收的消息。

- 一个对象能够做的事情
- 我们能够从一个对象取得的信息

- 声明格式

```
[public | protected | private]
[static][ final][abstract] [native] [synchronized]
返回类型  方法名([参数列表]) [throws exceptionList]
{
        方法体
}
```

- 格式说明
  - 方法修饰
    - public、protected、private 为存取控制符
    - static指明方法是一个类方法
    - final指明方法是一个终结方法
    - abstract指明方法是一个抽象方法
    - native用来集成java代码和其它语言的代码
    - synchronized用来控制多个并发线程对共享数据的访问

1.9.4 成员函数的定义

- 格式说明(续)
  - 返回类型
    - 方法返回值的类型，可以是任意的Java数据类型
    - 当不需要返回值时，返回类型为void
  - 参数类型
    - 简单数据类型，
    - 引用类型(数组、类或接口)
    - 可以有多个参数，也可以没有参数，方法声明时的参数称为形式参数
  - 方法体
    - 方法的实现
    - 包括局部变量的声明以及所有合法的Java指令
    - 局部变量的作用域只在该方法内部
  - throws exceptionList
    - 用来处理异常

◆方法调用

- ■给对象发消息意味着调用对象的某个方法
  - □从对象中取得信息
  - □修改对象的状态或进行某种操作
  - □进行计算及取得结果等
- ■调用格式

  <对象名>.<方法名>（[参数列表]）

  称点操作符“.”前面的<对象名>为消息的接收者(receiver)
- ■参数传递
  - □值传递：参数类型为基本数据类型时
  - □引用传递：参数类型为引用变量时（比如对象类型或数组时）

◆例如，假设有一个不带参数并返回 int 类型值的方法 f()。如果有一个可以调用 f()的对象a，可以按照如下格式调用：

■int x = a.f();

◆方法调用

■再比如，如下方法计算一个字符串占用的字节数

```
int storage(String s) {
  return s.length() * 2;
}
```

■s是一个字符串对象，一旦 s 被传递到方法中，就可以像对待任何其他对象一样对待它。这里调用了 length()方法，这是 Strings 的方法之一，它返回字符串中的字符数。

■其中的return表示方法执行结束。return后面可以跟上计算结果，返回给调用者。

■从前面这些例子可以看出：面向对象程序由大量的对象组成，这些对象可以将其他对象作为参数，并发送消息给其他对象

1.10 Java 的包

- Java 中的每个类都包含在称为包的东西中。未明确放入包中的类位于“默认”包中。
- Java的所有标准类都在包中，例如，像 String 和 System 这样的类，它们位于名为 java.lang 的包中。java.lang 中的类会自动导入到任何 Java 文件中，但是，其它包中的类必须使用import指令导入。
- 比如， ArrayList在java.util包中。为了使用ArrayList, 代码这样写：

```
import java.util.ArrayList;
public class SingleImport {
  public static void main(String[] args) {
    //如没有import java.util.ArrayList, 就必须写完全限定名
 java.util.ArrayList  ArrayList list = new java.util.ArrayList();
    //因为有import java.util.ArrayList, 直接写ArrayList也可以
    ArrayList list2 = new ArrayList();
  }
}
```

1.10 Java 的包

- 为了对java.util中其他类也能类似于ArrayList这样省略全名(或者完全限定名: full qualified name)，import时候可以用*代替类名：
  - import java.util.*;

1.10 Java 的包

- 利用包可以解决类名冲突的问题。不同包中的类可以同名。
- 如果在包中创建了一个类，则源代码的第一行就要指定该类在哪个包中。
- 例如，如果该类位于名为test.pkg 的包中，则源代码的第一行必须是

```
package test.pkg;
```

1.10 Java 的包

- 在命令行环境中使用包时，会出现一些复杂情况。
- 例如，如果程序位于名为 test.pkg 的包中，则源代码文件必须位于名为“test”的目录中名为“pkg”的子目录中，而该目录又位于 Java 主工作目录中。当编译或执行程序时，应该在主目录中工作，而不是在子目录中。
- 编译源代码文件时，必须在命令中包含目录名称：例如，对于 test.pkg 包中的程序，在 Linux上使用“javac test/pkg/ClassName.java”，在 Windows 上使用“javac test\pkg\ClassName.java”。
- 执行程序的命令是“java test.pkg.ClassName”，包名和类名之间用句点隔开。

◆1.11.1 Java 源代码文件的构成

- ■一个java文件称作一个编译单元。
- ■源文件的扩展名必须是 .java 。
- ■如果源文件中有多个类，则最多只能有一个public类
- ■如果有public类的话，那么源文件的名字必须与这个public类的名字相同（文件名的大小写可以与public类名的大小写不同）；
- ■如果源文件没有public类，那么源文件的名字由用户任意命名。
- ■如果该编译单元中还有其他类，由于它们不是公共的，则对于该包之外的程序是隐藏的
- ■编译.java 文件时，会为.java 文件中的每个类产生一个输出文件。每个输出文件在.java文件中都有一个类名，但扩展名为.class。

◆1.11.1 Java 源代码文件的构成

- ■在 Java 环境下，一个应用可能是一堆.class 文件，可以打包并压缩成一个 Java Archive (JAR)文件（使用 Java 的 jar 归档器来压缩）
- ■Java WEB程序可以打包为war文件。war文件的内容，除了一些jar或者.class文件之外，还有一些其他的应用需要的文件。
- ■不考虑注释，如果源代码文件的第一个语句是“package access;”，那么它说明编译单元是一个名为 access 的库，即，这个编译单元中的公共类的访问在 access 保护下，任何想要使用该类的程序都必须使用完全限定名或将 import 关键字与访问结合使用。
- ■Java开发时，利用import和package配合，解决名字冲突问题

1.11.2 包名与路径

- 一个包由许多.class 文件组成，这样容易引起混乱
- 使用操作系统的分层文件结构管理.class 文件。这是解决 Java编译单元中.class 引用杂乱的一种方式；另一种方式是使用jar文件（后面讲）
- 为了防止包名的冲突，包名必须唯一。因此，按照惯例，包名的第一部分推荐使用反向互联网域名。比如：
  - cn.edu.nwpu.cs
- 当运行 Java 程序时，将包名解析到你机器上的一个目录中，这样当 Java 程序需要加载.class 文件时，它可以找到.class 文件所在的目录。
- 环境变量CLASSPATH 包含一个或多个在搜索.class 文件时用作根的目录。
- 从该根目录开始，解释器将获取包名并用斜杠替换每个点以从 CLASSPATH 根目录生成路径名（因此包 foo.bar.baz 变为 foo\bar\baz 或 foo/bar/baz，两种表示由具体的操作系统决定）。然后将其连接到 CLASSPATH 中的各个条目。它会在那里查找名称与类对应的.class 文件。（它还搜索一些与 Java 解释器所在位置相关的标准目录。）

1.11.2 包名与路径

◆比如：List.java和Vector.java,如果第一条语句都是：

package cn.edu.nwpu.simple;

◆把这两个文件放入如下目录：

C:\DOC\Java\cn\edu\nwpu\simple

◆那么可以设置CLASSPATH类似如下：

CLASSPATH=.;D:\JAVA\LIB;C:\DOC\Java

◆CLASSPATH中如果包括jar文件，则必须写上jar文件的文件名：

CLASSPATH=.;D:\JAVA\LIB;C:\flavors\grape.jar

◆编译器处理import语句时，会根据CLASSPATH查找类。

1.11.3 冲突

- 如果通过“*”导入两个库并且它们包含相同的名称，会发生冲突问题.
  - 比如cn.edu.nwpu.simple和java.util都包含Vector类。
  - 代码利用*导入：

```
import cn.edu.nwpu.simple.*;
import java.util.*;
```

  - 一旦使用Vector类，就会导致编译错误，因为编译器不知道要用哪个Vector:

```
Vector v = new Vector();
```

- 使用限定名解决冲突
  - java.util.Vector v = new java.util.Vector();
- 或者import指定类名，不使用*

1.11.4 自定义工具库

- 有了以上知识，可以创建自己的工具库来减少或消除重复代码。
- 例如，考虑一下为System.out.println()使用的别名，以减少键盘输入代码的工作量。这可以是名为 Print 的类的一部分，以便最终获得可读的静态导入
- Java的静态导入（Static Import）是从Java 5开始引入的一个特性，它允许直接导入一个类的静态成员，而不必通过类名来引用这些成员。通常，在没有静态导入的情况下，你需要使用类名来引用静态方法或变量

1.11.4 自定义工具库

这就是“静态导入”

```
package cn.edu.nwpu.util;
import java.io.*;
public class Print {
// Print with a newline:
public static void print(Object obj) {
  System.out.println(obj);
}
// Print a newline by itself:
public static void print() {
  System.out.println();
}
// Print with no line break:
public static void printnb(Object obj)
{
  System.out.print(obj);
}
public static PrintStream printf(String
format, Object... args) {
  return System.out.printf(format,
args);
  }}
```

```
//: acces      ntTest.java
// Uses th      tatic printing
methods in      rint.java.
import static
cn.edu.nwpu.util.Print.*;
public class PrintTest {
public static void
main(String[] args) {
  print("Available from now
on!");
  print(100);
  print(100L);
  print(3.14159);
}
} /* Output:
Available from now on!
100
100
3.14159
*/
```

◆1.12.1 基本输出和格式化输出

- ■最基本的输出函数是 System.out.print(x)，其中 x 可以是任何类型的值或表达式。
- ■System.out.print(x);是不带换行的输出。
- ■System.out.printf类似于c语言的printf()。输出一些特殊符号时，使用以下格式：
  - □用%n输出换行。和\n不同的是，%n是平台无关的。
  - □要输出百分号，请在格式字符串中使用格式说明符%%。
  - □在字符串中使用反斜杠“\”来输出特殊字符，例如制表符和双引号字符。

1.12.2 文本输入例子

◆旧版的java读入用户输入是很麻烦的。在 Java 5.0 以后的版本，通过一个新的 Scanner 类使输入变得更容易了。

```
import java.util.Scanner;
public class Interest2WithScanner {
    public static void main(String[] args) {
        Scanner stdin = new Scanner( System.in ); // 创建一个输入类
        Scanner.
        double principal; // 投资金额
        double rate; // 年利率.
        double interest; // 一年的收益.
        System.out.print("Enter the initial investment: ");
        principal = stdin.nextDouble();
        System.out.print("Enter the annual interest rate (as a
        decimal): ");
        rate = stdin.nextDouble();
        interest = principal * rate; // 计算当年的收益.
        principal = principal + interest; // 累计投资金额.
        System.out.printf("The amount of interest is $%1.2f%n",
        interest);
        System.out.printf("The value after one year is $%1.2f%n",
        principal);
    } // end of main()
} // end of class Interest2WithScanner
```

1.12.2 文本输入例子

- stdin.nextDouble()、stdin.nextLong()和stdin.nextBoolean()。分别读取双精度浮点数、长整型以及布尔类型的数据。stdin.nextBoolean()只接受“true”或“false”作为输入。
- 查看Scanner类的帮助文档可以看到更多的输入方法。

Java程序的种类和结构

- ◆Java应用程序（Application）
  - ■它可以在Java平台上独立运行，通常称之为Java应用程序。Java应用程序是独立完整的程序，在命令行调用独立的解释器软件即可运行。
  - ■Java应用程序的主类必须有一个定义为 public static void main(String[] args) 的主方法，这个是Java应用程序执行的入口点
  - ■Java应用程序可划分为以下两种：
    - □命令行程序Command Line
    - □GUI (Graphics User Interface)
- ◆Java小程序（Applet）(Java 9开始已经废弃)
  - ■Applet是嵌入在HTML（超文本标记语言）文档中的Java程序，必须搭配浏览器来运行，因此称为小程序，也有人称为网页向导。
- ◆Java Servlet
  - ■应用于Web服务器
- ◆Android Application

◆Java程序注意事项：

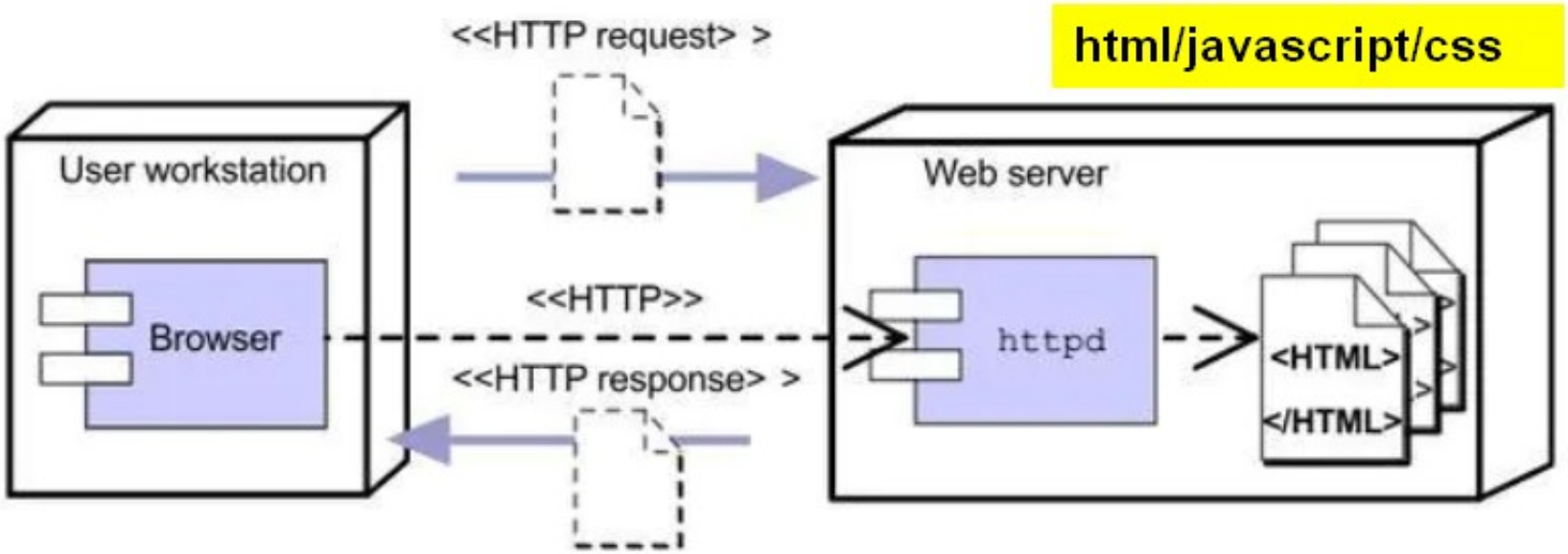
- ■Java是严格区分大小写的语言。
- ■一个程序中可以有多个类，但只能有一个类是主类。在Java应用程序中，这个主类是指包含main()方法的类。应用程序的主类不一定要求是public类。主类是Java程序执行的入口点。
- ■同一个Java程序中定义的若干类之间没有严格的逻辑关系要求，但它们通常是在一起协同工作的，每一个类都可能需要使用其他类中定义的静态属性或方法。

◆Java应用程序是由一系列的类定义构成的

- ■Java程序运行的过程，就是类的对象之间交互的过程。

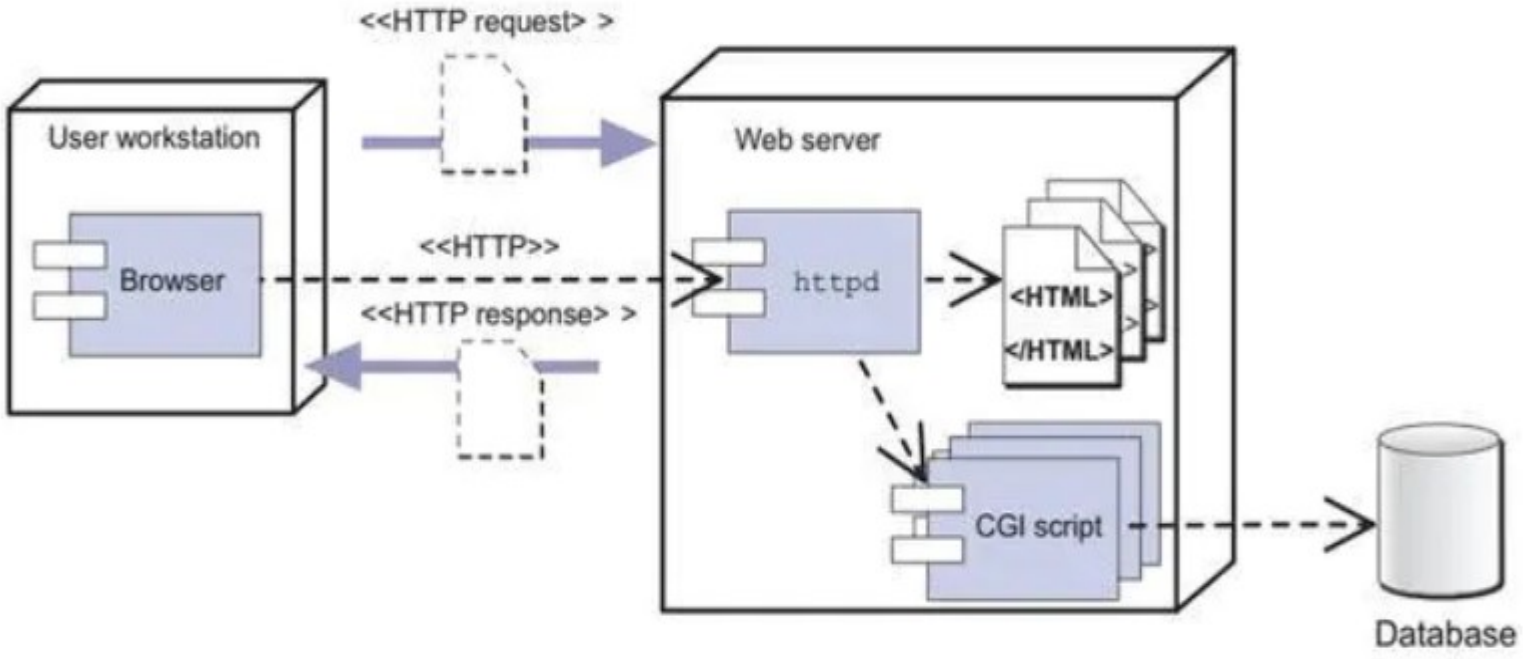
Java Web 简介（补充）

- https://www.jianshu.com/p/d9b770a78da1
- https://www.cnblogs.com/hellokitty1/p/4835408.html
- 最初的WEB:
  - WEB Server提供HTML/JavaScript/Css
  - 浏览器解释并展示结果。
  - 缺点：无法提供动态数据的展示。

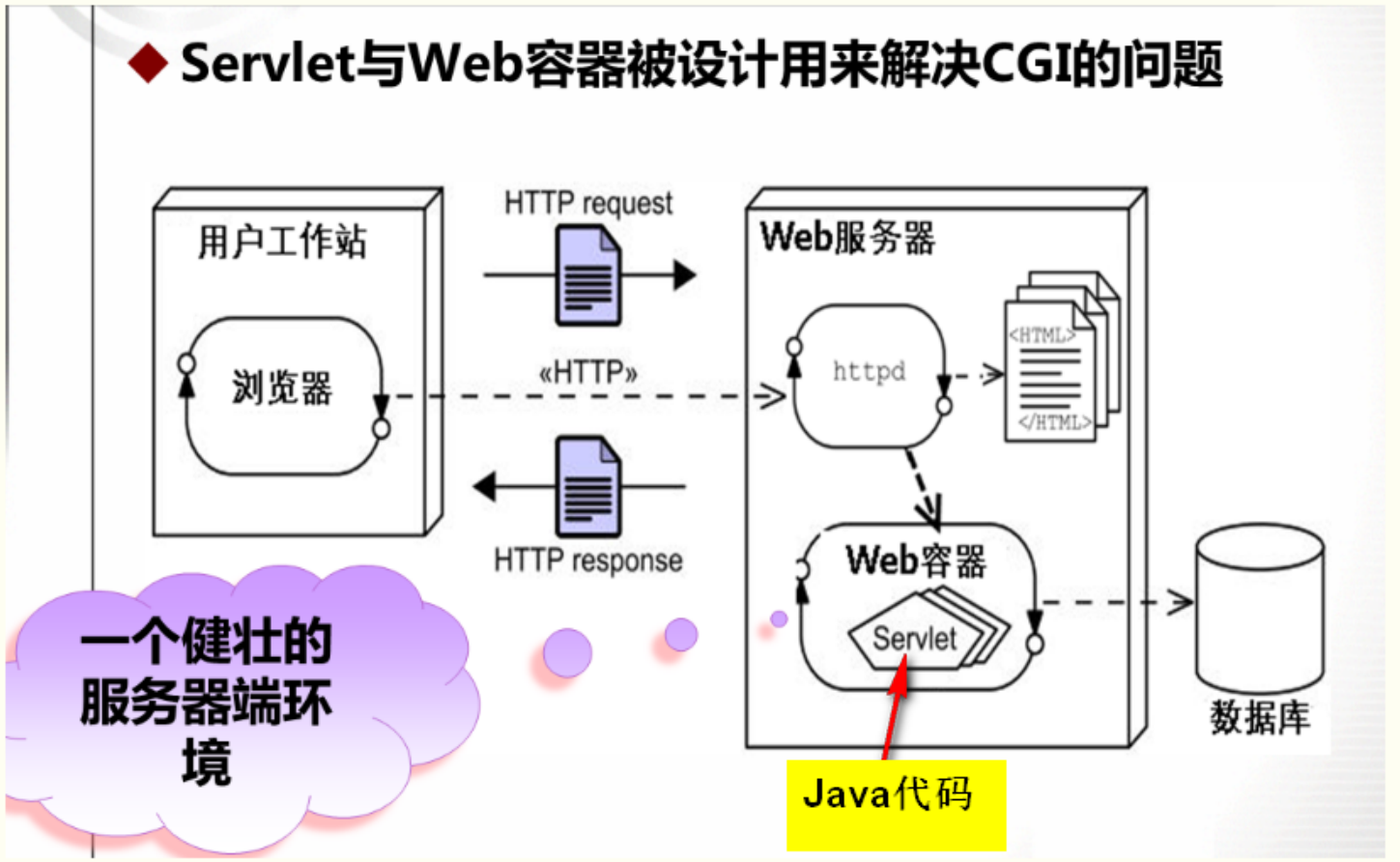


Java Web 简介

- 利用CGI(Common Gateway Interface)提供动态内容
- 动态内容被拼接成HTML/JavaScript/Css
- CGI程序可以是C, Perl, Python等任意语言（比如c语言可以用printf拼接需要的HTML/JavaScript/Css)
- 缺点:CGI以进程为单位执行，效率低下

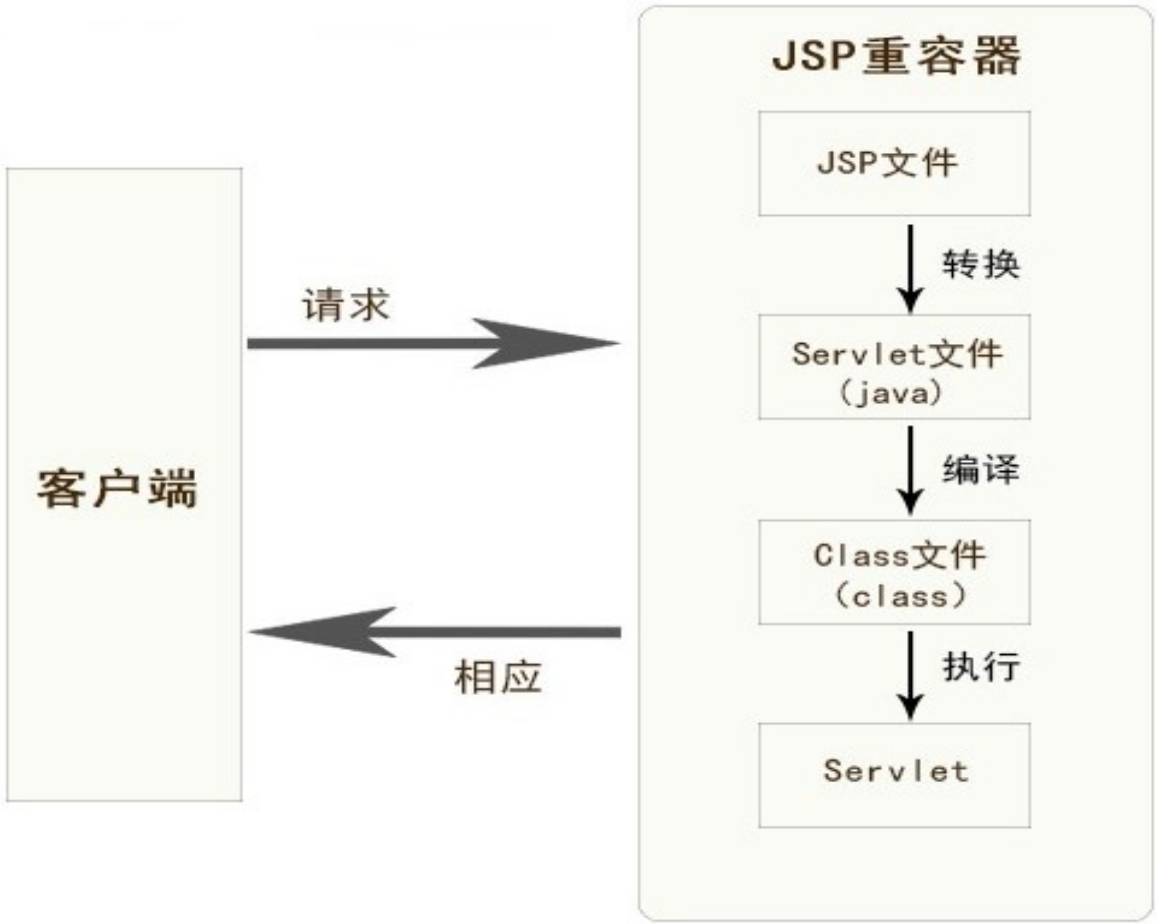


- Servlet以线程为单位，资源消耗少，并发能力提高
- 连接数据库使用JDBC. 目前已经被Mybatis等框架所包装。
- 使用system.out.println这样的方法拼接HTML/JavaScript/Css, 还是不方便。



- Jsp就是在HTML/JavaScript/Css中，混合Java代码。这样就不必用system.out.println来拼接输出了，方便编程
- ASP页面也是类似的原理(C#)
- Jsp在运行时

  编译为Servlet



Java Web简介

- Java WEB程序运行于WEB容器中。常用的容器
  - Tomcat
  - WegLogic
  - Netty(轻量级)
- 比如如下的JSP代码

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<html>
<head>
<title>JSP 测试</title>
</head>
<body>
<h1>JSP 测试!</h1>
<p> 机器名: <%= System.getenv("COMPUTERNAME") %> </p>
<p>系统当前时间: <%= new java.util.Date() %></p>
</body>
</html>
```

◆利用netty运行的脚本如下，运行后浏览器看到结果。（需要java17)




```
set JETTY_HOME=.\jetty-home-12.0.6
rmdir /s /q DemoJsp
mkdir DemoJSP
cd DemoJsp
java -jar ..\%JETTY_HOME%\start.jar  --add-modules=server,http,ee10-deploy,ee10-apache-jsp
mkdir webapps\root
copy ..\index.jsp webapps\root
java -Djetty.http.port=8088    -jar ..\%JETTY_HOME%\start.jar
```